

MaxProductOfThree: Algorithm Comparison

1. Algorithmic Approach & Mechanics

Non-Recursive (Iterative): The iterative algorithm computes the maximum product by performing a single linear pass over the array using a FOR loop [cite: 15, 21]. During each iteration, it updates five scalar variables (max1 , max2 , max3 and min1 , min2) by comparing the current element against them [cite: 19, 28]. After the loop, it calculates two potential products (product of the three maximums, or product of the two minimums and the single maximum) and returns the larger one [cite: 15, 29].

Recursive: The recursive approach performs the same logical comparisons but relies on a helper function (`FindExtremes`) [cite: 27]. Instead of a loop, it processes one element per function call, passing the updated extreme values (the max and min variables) alongside the incremented index in the recursion stack [cite: 13, 27]. The base case is reached when the index equals the length of the array [cite: 8, 27].

2. Time Complexity

Both algorithms perform a fixed set of operations per element regardless of the element's value, making their Best, Average, and Worst-case time complexities asymptotically identical [cite: 5, 6, 24, 25].

- **Non-Recursive:** Since the loop iterates exactly n times and performs a maximum of 5 comparisons per iteration, the time complexity is calculated as a summation: $C(n) = 5n$, which simplifies to an overall Time Complexity of $\Theta(n)$ [cite: 24, 25].
- **Recursive:** Modeled using the recurrence relation $T(n) = T(n-1) + c$, where c is the constant time taken for comparisons [cite: 9]. Expanding this via the iteration method down to the base case $T(0) = \Theta(1)$ ultimately yields a Time Complexity of $\Theta(n)$ [cite: 8, 10].

3. Space Complexity

This is where the two algorithms significantly diverge due to their structural implementation.

- **Non-Recursive:** Employs exactly five scalar variables to store the extremes, without requiring any auxiliary data structures [cite: 25, 26]. Consequently, its Space Complexity is $\Theta(1)$ (constant) [cite: 26].
- **Recursive:** Relies on the call stack to maintain state. It calls itself once for every element, producing a recursion depth equal to n [cite: 12]. Although each recursion frame only stores a constant number of variables, the cumulative depth results in a Space Complexity of $\Theta(n)$ [cite: 13].

4. Summary Comparison

Feature	Recursive Algorithm	Non-Recursive Algorithm
Basic Operation	Comparisons per frame [cite: 4]	At most 5 comparisons per loop iteration [cite: 22]
Time Complexity	$\Theta(n)$ [cite: 11]	$\Theta(n)$ [cite: 25]
Space Complexity	$\Theta(n)$ [cite: 13]	$\Theta(1)$ [cite: 26]
Dependency on Input	Does not vary [cite: 5]	Does not vary [cite: 20]
Memory Overhead	High (Recursion Stack Depth = n) [cite: 12]	Low (5 scalar variables) [cite: 25]

Conclusion

Both algorithms solve the MaxProductOfThree problem in optimal linear time ($\Theta(n)$) by employing a mathematical shortcut to avoid checking all possible triplets [cite: 10, 15, 25]. However, the **Non-Recursive** approach is vastly superior in terms of memory utilization, maintaining a constant $\Theta(1)$ space complexity compared to the Recursive algorithm's $\Theta(n)$ space requirements [cite: 13, 26].